# Microsoft Agent Framework and Microsoft Foundry Agent Service

A Detailed Guide | Prepared on November 10, 2026

*Both products evolve quickly, so features marked preview may have changed. Always confirm against Microsoft Learn.*

## 1. The Big Picture

Building an AI agent involves two separate problems:

1. **Writing the agent's logic:** which model it uses, which tools it can call, how several agents cooperate, and when a human must approve something.
2. **Running the agent in production:** hosting, scaling, identity, security, monitoring, memory, and governance.

Microsoft addresses these with two complementary products:

| Question | Product |
|---|---|
| How do I code my agents and workflows? | **Microsoft Agent Framework** (open-source SDK) |
| Where do I run, scale, and govern them? | **Microsoft Foundry Agent Service** (managed Azure platform) |

A simple analogy: Agent Framework is the engine design and the parts you assemble. Foundry Agent Service is the garage, fuel, insurance, and dashboard that let the vehicle run safely on a public road.

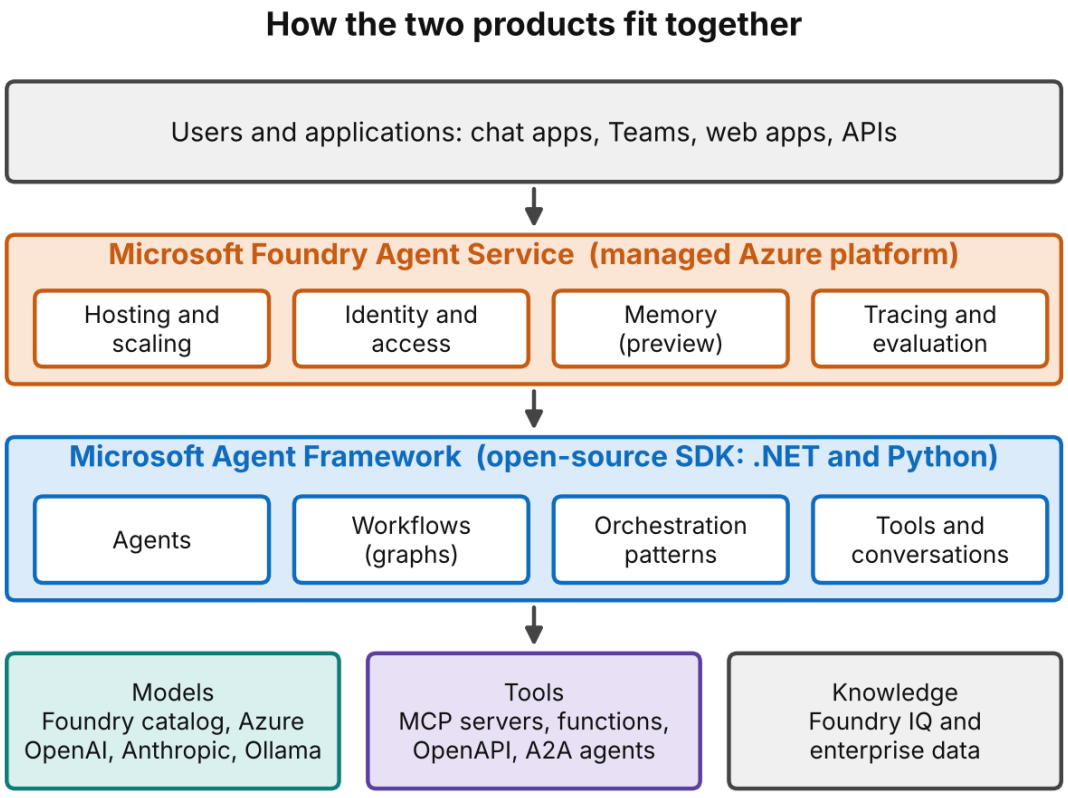



*Figure 1. The two products sit at different layers of the same stack.*

# 2. Microsoft Agent Framework

## 2.1 What it is

Microsoft Agent Framework is an open-source development kit for building AI agents and multi-agent workflows in **.NET and Python**. It is open source on GitHub, so the same code can move from prototype to production, across model providers and across deployment targets.

## 2.2 Where it came from

Agent Framework is the direct successor to two earlier Microsoft projects, created by the same teams:

- **AutoGen** (from Microsoft Research) was known for simple, intuitive multi-agent abstractions and conversation patterns.
- **Semantic Kernel** was known for enterprise-grade features such as state management, type safety, middleware, and telemetry.

Agent Framework merges the two. It keeps AutoGen's easy agent abstractions and orchestration ideas, adds Semantic Kernel's enterprise robustness, and layers explicit multi-agent orchestration on top. One practical difference from Semantic Kernel is that you can create an agent directly, without first coupling it to a Kernel object.

## 2.3 Why it exists

- **One framework instead of many.** Developers no longer have to stitch together several agent libraries.
- **Production readiness.** It includes observability, security, and durability from the start.
- **Interoperability.** It supports open standards (MCP, A2A, OpenAPI) so you are not locked into one vendor.
- **Future-proofing.** It is model-agnostic, so you can change providers as AI models evolve.

## 2.4 The two core building blocks

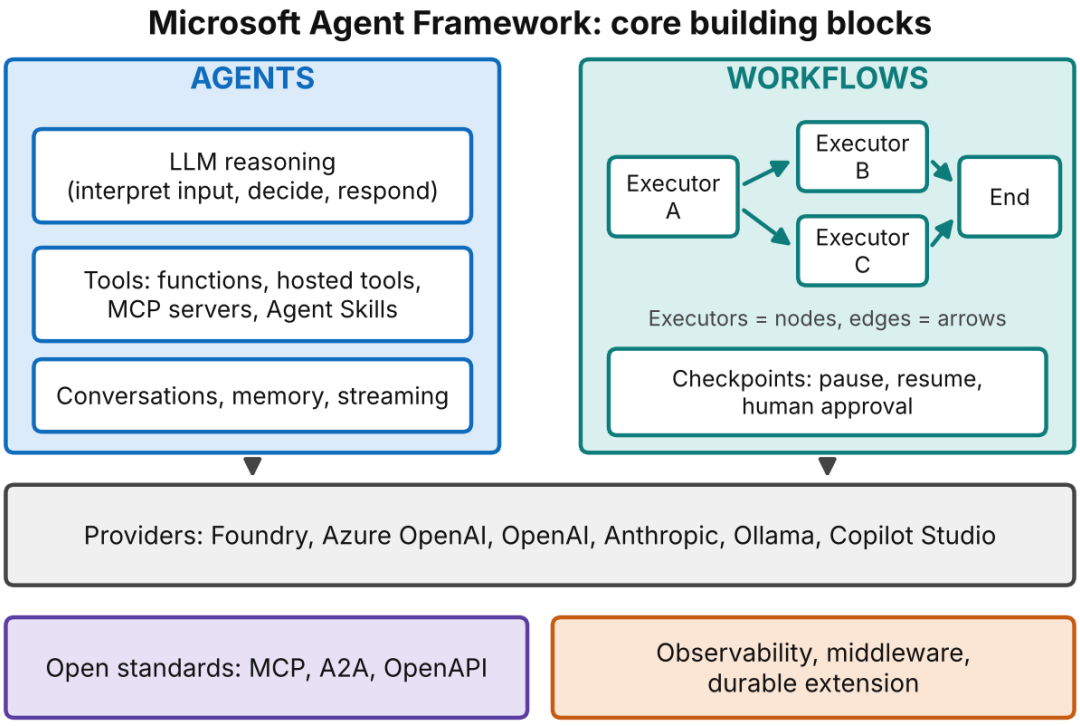



*Figure 2. Agents and workflows are the two core building blocks, sitting on providers and open standards.*

### *Agents*

An agent uses a large language model to interpret input, decide what to do, call tools (including MCP servers), and generate a response. Agents are best for tasks that are unstructured and cannot be fully defined in advance, because LLM behavior is probabilistic.

Agent Framework gives agents a consistent interface regardless of provider. Supported providers include Microsoft Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama (for local models), and Microsoft Copilot Studio agents.

Key agent features covered by the documentation:

- **Tools:** custom functions, hosted tools, and MCP servers that the agent can call.
- **Agent Skills:** reusable bundles of capability.
- **Conversations:** multi-turn dialogue with history.
- **Memory and persistence:** keeping context across turns and sessions.
- **Streaming:** real-time output so users see responses as they are generated.
- **Human-in-the-loop:** approval workflows and intervention points, for example requiring a person to approve a refund before the agent executes it.

### *Workflows*

When a single agent is not enough, or when you need predictable control, you use workflows. A workflow models a multi-step process as a **directed graph**:

- **Executors** are the nodes. Each one does a unit of work, which can be an agent, a function, or a human step.

- **Edges** are the connections that decide how data and control flow from one executor to the next, including conditional routing.
- **Workflow Builder** is the API you use to assemble the graph and run it.
- **Checkpointing** saves progress so long-running processes can pause, survive failures, wait for human input, and resume.

The key distinction: **agents provide flexible reasoning, while workflows provide explicit, controllable structure.** Many real systems combine both, with deterministic workflow steps and agents embedded where judgment is needed.

## 2.5 Multi-agent orchestration patterns

Drawing on AutoGen's heritage, the framework supports these ready-made patterns:

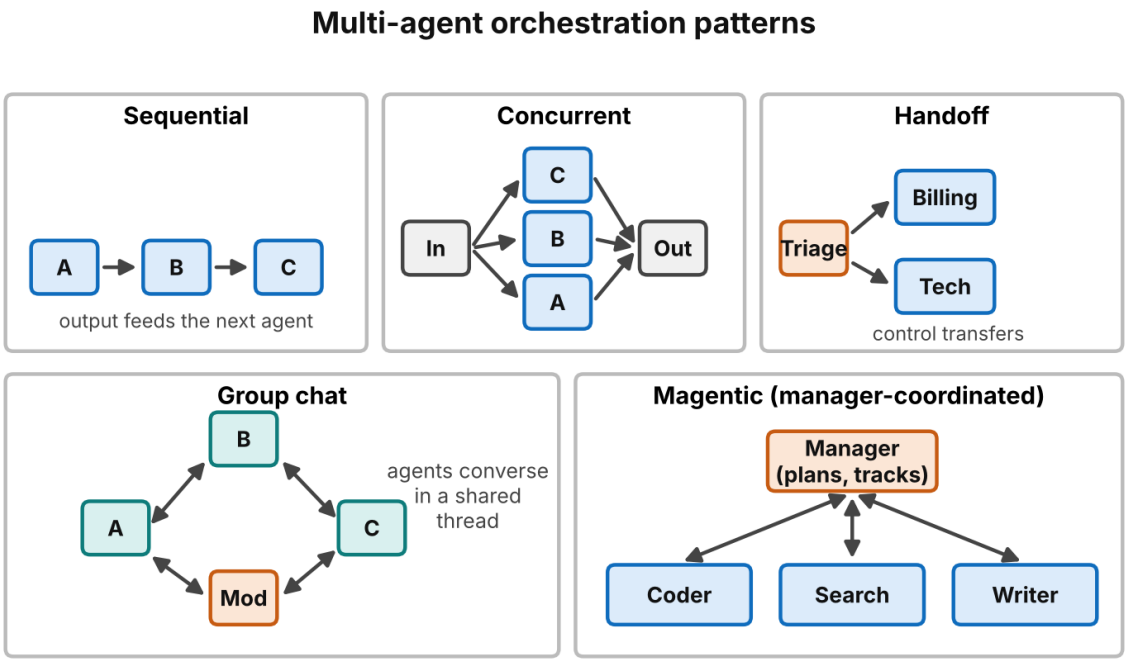



*Figure 3. The five orchestration patterns at a glance.*

| Pattern | How it works | Typical use |
|---|---|---|
| **Sequential** | Agents run one after another; each receives the previous output. | Draft, then review, then polish pipelines |
| **Concurrent (parallel)** | Several agents work on the same input at once and results are combined. | Getting multiple specialist opinions quickly |
| **Handoff** | One agent transfers control to another better suited to the request. | Triage agent routing to billing or technical support |
| **Group chat** | Several agents converse collaboratively, often with a moderator. | Brainstorming, debate, collaborative problem solving |
| **Magentic** | A manager agent plans the task and coordinates specialist agents dynamically. | Open-ended, complex tasks with no fixed plan |

## 2.6 Open standards and interoperability

- **Model Context Protocol (MCP):** a standard way for agents to reach tools and data sources.
- **Agent-to-Agent (A2A):** a standard for agents, possibly built with different frameworks, to communicate with each other.
- **OpenAPI:** lets agents call existing REST APIs described by an OpenAPI spec.
- **Microsoft 365 Agents SDK and Foundry integrations:** connect agents to enterprise surfaces and the managed Foundry Agent Service.

## 2.7 Getting started

Python installation options:

- Core framework: `pip install agent-framework`
- With Azure integrations: `pip install agent-framework[azure]`
- With OpenAI integration: `pip install agent-framework[openai]`
- Everything: `pip install agent-framework[all]`

The official learning path goes in this order:

1. Build your first agent.
2. Add tools.
3. Hold multi-turn conversations.
4. Add memory and persistence.
5. Build workflows.
6. Host your agent.

**Important:** the Python package layout was reorganized in version 1.2.0 (April 2026), so older tutorials may use outdated imports. Check the current documentation and samples on GitHub, which cover both Python and C#.

## 2.8 When to choose Agent Framework

Choose it when you:

- standardize on .NET or Python and use Azure or Microsoft developer tooling;
- need explicit workflow control with clear coordination of steps;
- have long-running or human-in-the-loop processes that need checkpoints;
- want a single framework rather than several agent libraries;
- want portability across model providers and hosting locations.

## 2.9 Strengths and cautions

| Strengths | Cautions |
|---|---|
| Open source with an active GitHub project | Relatively new; APIs have changed between versions |
| Unifies Semantic Kernel and AutoGen ideas | Older Semantic Kernel or AutoGen code needs migration |
| Graph workflows with checkpointing | You still must host and operate it yourself unless you use a managed service |
| Provider-agnostic and standards-based | Probabilistic agents still need evaluation and guardrails |

# 3. Microsoft Foundry Agent Service

## 3.1 What it is

Microsoft Foundry Agent Service is a **fully managed service** for building, deploying, and scaling AI agents securely, without managing the underlying compute and storage. It connects the core pieces of Foundry (models, tools, and frameworks) into a single runtime, and is the path from prototype to a governed, identity-scoped production deployment on Azure.

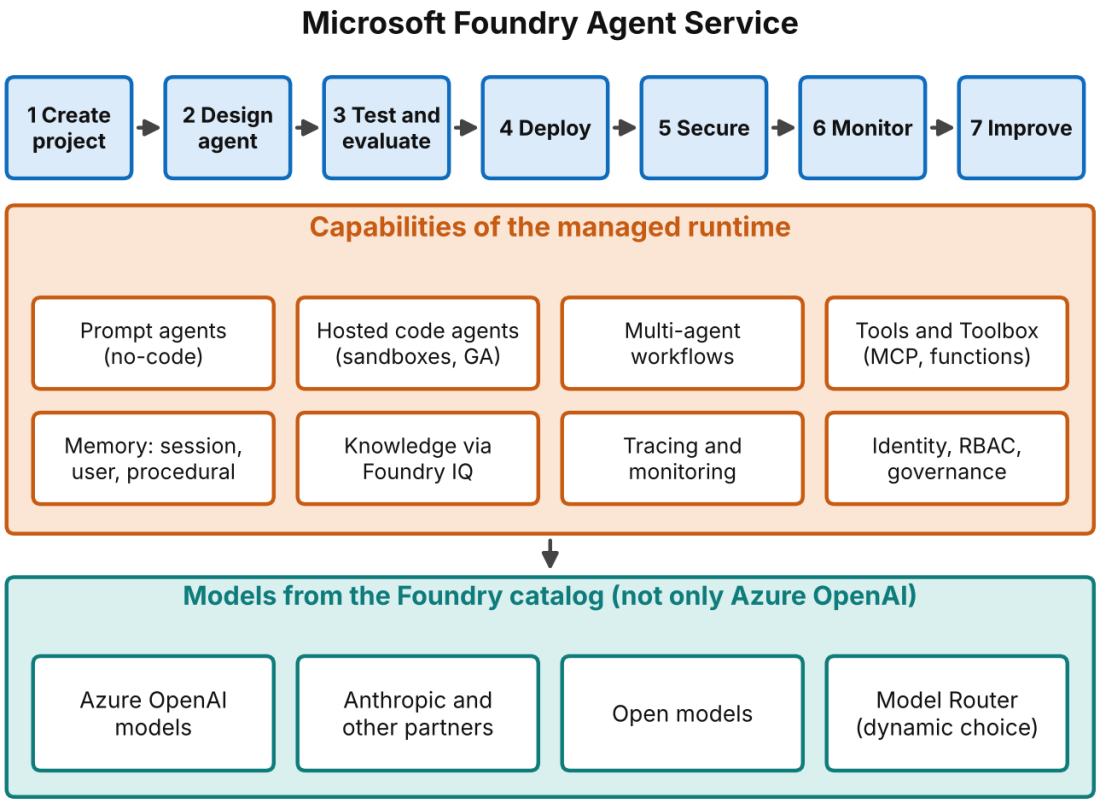



*Figure 4. Foundry Agent Service: the agent lifecycle, the managed runtime capabilities, and the model catalog underneath.*

## 3.2 Naming and status

- Azure AI Foundry was rebranded **Microsoft Foundry** around Build 2026 (May 2026). Older articles and screenshots may still say Azure AI Foundry or Azure AI Agent Service. Role names such as Azure AI User became Foundry User.
- Foundry Agent Service reached **general availability in March 2026**.
- Hosted agents reached general availability in **July 2026**.
- Some newer capabilities, such as memory, toolbox, tracing for hosted agents, and the agent optimizer, were in **preview** at the time of the sources used here.

## 3.3 Position inside Microsoft Foundry

Microsoft Foundry is a broader platform that brings together models, agents, knowledge, evaluation, and deployment. Within it:

- **Model catalog:** thousands of models, with a Model Router that can dynamically choose models by use case, cost, and quality.
- **Foundry Agent Service:** the agent runtime and management layer (this document's focus).
- **Foundry IQ:** the knowledge layer that grounds agents in enterprise data (generally available since June 2026).

- **Evaluation, tracing, and governance tools:** for testing and monitoring agents.

## 3.4 Key capabilities

### *Fully managed runtime*

You do not provision servers or storage for your agents. Microsoft handles hosting, scaling, and reliability, and the service integrates with Azure identity and security.

### *Choice of models*

The service works with many models from the Foundry model catalog, not only Azure OpenAI models. That lets teams balance quality, cost, and latency per agent.

### *Multiple ways to build agents*

- **Prompt agents (no-code or low-code):** create an agent in the Foundry portal using instructions, a model, and tools, often starting from built-in templates.
- **Hosted (code-based) agents:** bring agents written in code, including with Microsoft Agent Framework or other frameworks, and run them on Foundry inside secure sandboxes.
- **Multi-agent workflows:** coordinate several agents, with support for scenarios such as voice interaction through Voice Live.

### *Tools*

Tools are what turn a chat model into an agent that acts. Foundry supports built-in tools, custom functions, and MCP-based tools. A **Toolbox** feature (preview) provides unified tool management so teams can govern and reuse tools centrally.

### *Memory*

Agents need to remember. Memory in Foundry Agent Service (public preview as of June 2026) includes:

- **Session memory:** context within a single conversation.
- **User memory:** facts and preferences about a specific user across sessions.
- **Procedural memory:** learned ways of doing tasks.

This removes the need to build and operate your own external memory database.

### *Knowledge and grounding*

With Foundry IQ, agents can be grounded in enterprise data sources through a unified grounding approach, so answers rely on company information rather than only the model's training data.

### *Observability, evaluation, and governance*

- **Tracing:** view session details, run steps, and tool calls to debug behavior and performance (hosted-agent tracing in preview).
- **Monitoring dashboards:** track agent health and usage.
- **Evaluation:** test agent quality before and after changes.
- **Red teaming:** an AI Red Teaming Agent exists for safety testing.
- **Identity and access:** each agent can be scoped with Azure identity and role-based access control.
- **Agent optimizer (preview):** helps improve agent performance.

## 3.5 Typical lifecycle of an agent on Foundry

1. **Create** an Azure subscription and a Foundry project.
2. **Design** the agent: pick a model, write instructions, and attach tools and knowledge.
3. **Test** in the playground and run evaluations.

4. **Deploy** the agent (prompt agent or hosted code agent).
5. **Secure** it with identity, permissions, and content safety controls.
6. **Monitor** with tracing and dashboards.
7. **Improve** using evaluation results and optimization, then republish.

## 3.6 Getting started

1. Sign up for an Azure subscription and open the Microsoft Foundry portal.
2. Create a project and deploy a model.
3. Create your first prompt agent using the portal's no-code interface and templates.
4. Add tools or knowledge, and test it.
5. When you need custom logic, write a code-based agent (for example with Agent Framework) and host it on the service.

The official documentation is continuously updated, and the overview carries an August 2026 update stamp. Expect roughly three to four hours to work through the overview, concepts, and quickstarts.

## 3.7 When to choose Foundry Agent Service

Choose it when you:

- want production agents without building hosting, identity, observability, and security yourself;
- are an enterprise that needs governance, auditability, and Azure integration;
- want to mix many models from the catalog;
- need built-in memory and tool management;
- want to move quickly from a no-code prototype to a governed deployment.

## 3.8 Strengths and cautions

| Strengths | Cautions |
|---|---|
| Managed, scalable, Azure-integrated | Tied to Azure and its pricing model |
| Strong governance and identity story | Several features still in preview |
| Supports prompt agents and hosted code agents | Naming and portal changed with the rebrand |
| Built-in memory, tools, and tracing | Costs must be monitored (models, tools, storage) |

## 4. How the Two Fit Together

The products are not competitors. They sit at different layers (see Figure 1):

| | Agent Framework | Foundry Agent Service |
|---|---|---|
| Nature | Open-source SDK (code library) | Managed Azure platform |
| Main purpose | Define agent logic, tools, and workflows | Host, scale, secure, observe, and govern agents |
| Where it runs | Anywhere: laptop, your servers, any cloud | On Microsoft-managed infrastructure in Azure |
| Languages | .NET and Python | Portal, REST API, and SDKs |
| Orchestration | Sequential, concurrent, handoff, group chat, magentic, graph workflows | Multi-agent workflows through the service, and hosts code that uses Agent Framework |
| Memory | Conversation memory and persistence in code | Managed session, user, and procedural memory (preview) |
| Governance | You build or integrate it | Built in (identity, tracing, evaluation) |
| Cost model | Free library; you pay for your own hosting and model usage | Azure consumption pricing |

**Common architecture:**

1. Write agents and workflows with Agent Framework.
2. Use Foundry models and tools through its provider integrations.
3. Deploy the result as a hosted agent on Foundry Agent Service.
4. Use Foundry tracing, evaluation, and identity controls to operate it.

## 5. Decision Guide

| Your situation | Suggested approach |
|---|---|
| Learning or prototyping a multi-agent idea in code | Agent Framework locally |
| Business team wants a simple assistant quickly | Foundry prompt agent (no-code) |
| Enterprise needing governed, scalable production agents | Agent Framework plus Foundry Agent Service hosting |
| Need to run on-premises or in another cloud | Agent Framework self-hosted, possibly with other model providers |
| Migrating existing Semantic Kernel or AutoGen apps | Plan a migration to Agent Framework, then host where needed |